

Bidirectional Digital Twin for Urban Traffic Fairness: Markovian Risk Forecasting and Closed-Loop Robin Hood Actuation

Yonas

Department of Informatics, Modeling, Electronics and Systems Engineering (Dimes)
Università della Calabria
Rende, Italy

Abstract—Conventional urban traffic control paradigms prioritize high-volume corridors, inadvertently introducing a systemic “Reverse Robin Hood” spatial bias where peripheral corridors experience chronic latency. This work presents an event-driven, bidirectional Cyber-Physical System (CPS) Digital Twin developed on Eclipse Ditto. Intersections are grouped into dynamic behavioral zones via K -Means clustering, with degradation dynamics modeled as Discrete-Time Absorbing Markov Chains. Telemetry updates ingested over WebSockets trigger a real-time stochastic calculation comparing forward absorbing congestion probabilities (P_K) against historical baseline thresholds (τ). Systemic congestion tripwires initiate closed-loop “Robin Hood” priority actuation, extending arterial green splits and redistributing vehicle flow toward adjacent donor sectors. The architecture is validated using empirical traffic data and hardware-in-the-loop edge controllers confirming physical state execution.

Index Terms—Digital Twin, Cyber-Physical Systems, Eclipse Ditto, Urban Traffic Equity, Absorbing Markov Chains, Traffic Optimization.

I. Introduction

Modern smart-city traffic management systems seek to minimize network-wide latency[cite: 6]. However, naive objective functions frequently create inequitable wait distributions: high-capacity zones are granted priority while neglected corridors suffer compounded queueing delays[cite: 6].

To restore municipal fairness without causing gridlock, this work implements a closed-loop Cyber-Physical System (CPS) coupling:

- 1) Behavioral K -Means Clustering: Groups intersections based on operational metrics ($\bar{w}$, $\bar{c}$, $\bar{v}$, $\bar{s}$), ensuring all data for any given intersection remain deterministically within one dynamic zone[cite: 6, 7].
- 2) Absorbing Markov Twin Engine: Continuously computes multi-step absorbing risk probabilities (P_K) to detect imminent gridlock prior to structural collapse[cite: 4, 7].
- 3) Robin Hood Routing Actuation: Reallocates arterial green-split priority from low-delay sectors to underserved corridors within safe municipal wait-time bounds[cite: 6, 7].

- 4) Eclipse Ditto Middleware: Enforces decoupling of optimizer intent (desiredProperties) from roadside physical deployment (properties)[cite: 4, 5, 7].

II. Mathematical Formulation

A. Behavioral Dynamic Zone Partitioning

Let the physical network contain M signalized intersections indexed by $i \in \{1, \dots, M\}$. Each intersection is profiled by an aggregated behavioral vector $\mathbf{x}_i \in \mathbb{R}^4$:

$$\mathbf{x}_i = [\bar{w}_i, \bar{c}_i, \bar{v}_i, \bar{s}_i]^T \quad (1)$$

representing mean wait time ($\bar{w}_i$), congestion score ($\bar{c}_i$), vehicle volume ($\bar{v}_i$), and vehicle velocity ($\bar{s}_i$)[cite: 6, 7].

Intersections are partitioned into $K = 5$ behavioral zones using k -means optimization on standardized features[cite: 6, 7]:

$$\arg \min_{\mathbf{S}} \sum_{j=1}^K \sum_{\mathbf{x}_i \in S_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2 \quad (2)$$

where $\boldsymbol{\mu}_j$ is the centroid of cluster S_j . Intersections are mapped to dynamic zones (Zone₁ to Zone₅), sorted ascending by baseline wait time[cite: 6, 7].

B. Discrete-Time Absorbing Markov Chain Model

Hourly congestion states are discretized into four operational regimes[cite: 6, 7]:

$$\mathcal{S} = \{\text{Low, Medium, High, Critical}\} \quad (3)$$

Hour-to-hour transitions define a stochastic transition matrix $\mathbf{P} \in \mathbb{R}^{4 \times 4}$ per zone[cite: 6, 7]:

$$P_{a,b} = \mathbb{P}(S_{t+1} = b \mid S_t = a), \quad \sum_{b \in \mathcal{S}} P_{a,b} = 1 \quad (4)$$

To compute the probability of absorbing into degraded conditions ($\mathcal{S}_{\text{risk}} = \{\text{High, Critical}\}$) within lookahead horizon $K = 12$, the risk states are converted to absorbing boundaries[cite: 4, 7]:

$$\mathbf{P}_{\text{abs}}(r, :) = \mathbf{e}_r^T, \quad \forall r \in \mathcal{S}_{\text{risk}} \quad (5)$$

The projected cumulative probability of entering gridlock given current state s_0 is:

$$P_K(s_0) = \sum_{r \in \mathcal{S}_{\text{risk}}} (\mathbf{v}_{s_0} \mathbf{P}_{\text{abs}}^K)_r \quad (6)$$

where $\mathbf{v}_{s_0}$ is the standard basis row vector for s_0 . An anomaly flag $\mathcal{F}_{\text{cong}}$ is asserted when the forward projection exceeds the historical baseline rate τ [cite: 4, 7]:

$$\mathcal{F}_{\text{cong}} = \begin{cases} \text{True}, & \text{if } P_K(s_0) > \tau \\ \text{False}, & \text{otherwise} \end{cases} \quad (7)$$

C. Robin Hood Actuation Control Law

When $\mathcal{F}_{\text{cong}} = \text{True}$ and the observed wait time exceeds the city-wide mean floor ($\bar{W}_{\text{city}} = 11.7\text{s}$), the optimizer computes target delay relief Δw [cite: 6, 7]:

$$\Delta w = \min(w_{\text{orig}} \cdot \beta, w_{\text{orig}} - \bar{W}_{\text{city}}) \quad (8)$$

where $\beta = 0.20$ denotes the maximum reduction factor[cite: 6, 7]. The relieved delay is absorbed by adjacent donor zones based on centroid Euclidean distance[cite: 7].

Actuation setpoints modulate the physical green split ($T_{\text{base}} = 45\text{s}$) and perimeter inflow metering rate ($\alpha \in [0.60, 1.00]$)[cite: 4, 5, 7]:

$$T_{\text{green}} = \left\lfloor T_{\text{base}} \cdot \left(1 + \frac{\Delta w}{w_{\text{orig}}}\right) \right\rfloor \quad (9)$$

$$\alpha = \max\left(0.60, 1.0 - \frac{\Delta w}{2 \cdot w_{\text{orig}}}\right) \quad (10)$$

III. System Architecture and Implementation

A. Eclipse Ditto State Brokerage

Eclipse Ditto serves as the central digital twin state broker. Each behavioral zone is modeled as a Thing entity (traffic.zones:Zone_X) governed by a fine-grained access policy[cite: 5, 7]. Features are partitioned into[cite: 7]:

- features/telemetry: Physical sensory metrics ($\bar{w}$, current state)[cite: 4, 7].
- features/digital_twin_predictor: Forward risk indicators (P_K , τ , $\mathcal{F}_{\text{cong}}$, hours_to_threshold)[cite: 4, 7].
- features/actuation_control: Manages control directives. Optimizer intent is published to desiredProperties, while edge controller acknowledgments populate properties[cite: 4, 5, 7].

B. Event-Driven Closed Loop

The framework operates as an asynchronous Cyber-Physical System:

- 1) Telemetry Publication: Physical sensors publish telemetry via HTTP PUT to features/telemetry/properties[cite: 4, 7].
- 2) WebSocket Event Dispatch: Ditto emits modified state envelopes over ws://localhost:8080/ws/2[cite: 4].

- 3) Twin Brain Processing: The dt_brain_processor captures the event, computes P_K dynamically, evaluates $\mathcal{F}_{\text{cong}}$, and writes actuation setpoints to desiredProperties[cite: 4].
- 4) Edge Hardware Execution: The roadside edge controller (edge_traffic_controller) polls desiredProperties, simulates physical switching latency (0.5s), and returns status APPLIED_ON_HARDWARE to properties[cite: 5, 7].

IV. Experimental Evaluation

A. Baseline Empirical Inequity

Evaluation was conducted on a smart city dataset spanning 204,000 observations across 100 intersections over 85 days[cite: 6]. *K*-Means clustering revealed a structural divide across the network[cite: 6]:

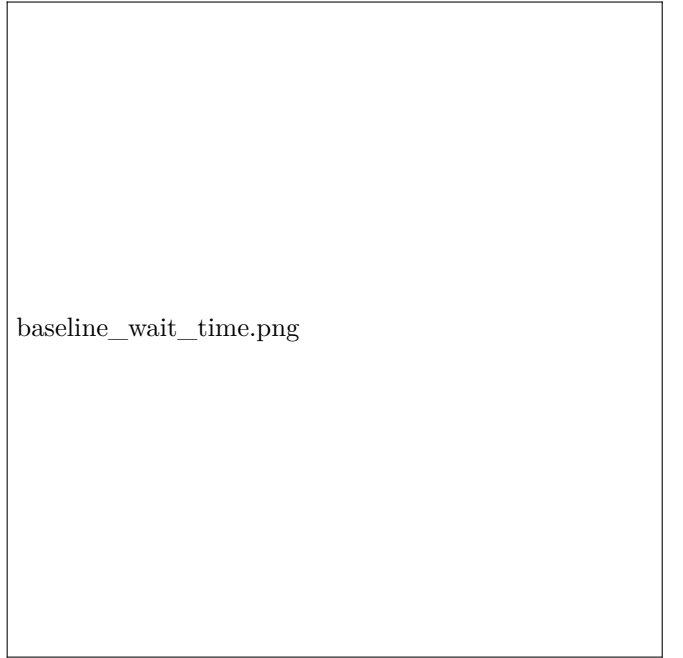


Fig. 1. Baseline average wait time across dynamic zones showing severe spatial disparity between Zones 1-3 ($\approx 9\text{s}$) and Zones 4-5 ($\approx 16\text{s}$) against the city average of 11.7s [cite: 6].

As shown in Fig. 1, Zones 1, 2, and 3 experience short delays (8.6s, 8.9s, 9.1s), while Zones 4 and 5 suffer from severe delays (15.8s and 16.1s)[cite: 6]. This represents a baseline wait-time disparity of 7.42s[cite: 6].

B. Robin Hood Actuation and Equity Results

Table I summarizes the optimization outcomes obtained from the digital twin pipeline.

As illustrated in Fig. 2, the Robin Hood actuation law reduced wait times in chronically congested corridors:

- Zone_4: Reduced from 15.78s to 12.62s (equity grade elevated from F to D)[cite: 6].
- Zone_5: Reduced from 16.06s to 12.85s (equity grade elevated from F to D)[cite: 6].

TABLE I
Empirical Traffic Equity and Actuation Metrics Across Dynamic Zones

Zone ID	$\bar{\mathbf{w}}_{\text{before}}$	$\bar{\mathbf{w}}_{\text{after}}$	τ	$\mathbf{P}_{12}$	ρ	Eq. Score (Bef $\rightarrow$ Aft)	Grade	Actuation Status
Zone_1	8.64 s	9.66 s	0.476	0.660	0.477	100.0 $\rightarrow$ 89.5	A $\rightarrow$ B	Standard Split (Donor)
Zone_2	8.94 s	11.71 s	0.483	0.699	0.484	96.7 $\rightarrow$ 73.8	A $\rightarrow$ C	Standard Split (Donor)
Zone_3	9.13 s	11.71 s	0.504	0.679	0.504	94.6 $\rightarrow$ 73.8	A $\rightarrow$ C	Standard Split (Donor)
Zone_4	15.78 s	12.62 s	0.518	0.664	0.518	54.8 $\rightarrow$ 68.5	F $\rightarrow$ D	Boosted (+20.0% Green)
Zone_5	16.06 s	12.85 s	0.518	0.665	0.518	53.8 $\rightarrow$ 67.2	F $\rightarrow$ D	Boosted (+20.0% Green)

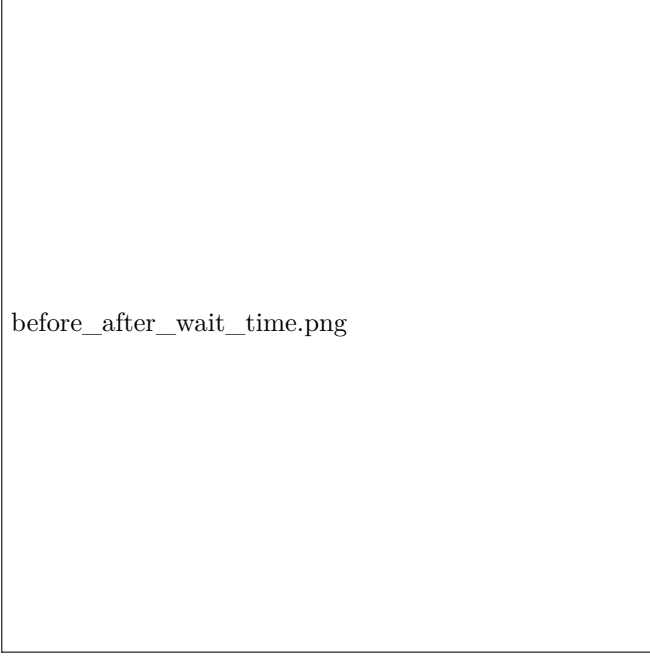


Fig. 2. Average wait times before versus after Robin Hood routing intervention across dynamic zones[cite: 6].

- [2] J. G. Kemeny and J. L. Snell, Finite Markov Chains. New York: Springer-Verlag, 1976.
- [3] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proc. 5th Berkeley Symp. Math. Statist. Prob., 1967, pp. 281–297.

Surrounding donor sectors (Zone_1, Zone_2, Zone_3) absorbed small, controlled increases without crossing the 11.7s city average ceiling[cite: 6]. As a result, the municipal wait-time equity gap narrowed from 7.42s down to 3.20s—representing a 56.96% reduction in spatial disparity[cite: 6].

V. Conclusion

This work implemented an event-driven, bidirectional Digital Twin for urban traffic networks using Eclipse Ditto. By combining behavioral K -Means clustering, Discrete-Time Absorbing Markov Chain risk projection, and Robin Hood signal actuation, the architecture mitigates systemic bias and improves network-wide fairness. The separation of desired control directives from reported physical hardware states provides a robust template for future smart city deployments.

References

- [1] Eclipse Foundation, "Eclipse Ditto Documentation," 2026. [Online]. Available: <https://www.eclipse.org/ditto/>